Prompt for continuous lines on calendar

I have a fleet management PWA called INYATHI. The calendar currently renders coloured stripes inside each day cell to show active bookings. The stripes are correct but they break at every cell border, leaving a visual gap between days. I want the stripes to appear as continuous bars that flow through from the booking start date to the booking end date with no gaps, like Google Calendar's multi-day event bars.

Nothing outside the calendar rendering should change.

== HOW THE CALENDAR CURRENTLY WORKS ==

renderCalendar() in admin.js builds a 7-column CSS grid (#cal-grid).
It first renders 7 .cal-day-name header cells, then one .cal-day div per day of the month, with empty filler divs for days before the 1st.
Each .cal-day contains:
- a .day-number div
- a .booking-markers div containing one .booking-marker div per booking

The coloured stripes are .booking-marker elements styled as:
display: block; width: 100%; height: 5px; border-radius: 2px;

The gap appears because each stripe is clipped inside its own .cal-day cell with its own padding, so stripes never visually connect across cells.

== THE APPROACH ==

Instead of rendering stripes inside individual day cells, render them as absolutely positioned bars on a wrapper that spans the full calendar grid. This means:

1. Wrap the existing #cal-grid div in a new relatively-positioned container.
2. After the day cells are built, calculate the pixel position and width of each booking bar by reading the grid layout, then absolutely position coloured bars behind the day numbers.
3. Stripes for bookings that span multiple days will visually connect across cells with no gap.

== EXACT IMPLEMENTATION ==

STEP 1 — In index.html, find the calendar card section. The cal-grid div currently looks like:

```
<div class="cal-grid" id="cal-grid"></div>
```

Wrap it in a relative container:

```
<div style="position:relative" id="cal-grid-wrapper">
  <div class="cal-grid" id="cal-grid"></div>
</div>
```

STEP 2 — In style.css, update .cal-day so it has no padding on the stripe area, and ensure the day number sits above the stripe layer:

```
.cal-day {
  min-height: 52px;
  border-radius: 4px;
  position: relative;
  display: flex;
  flex-direction: column;
```

```
  padding: 4px 3px 3px;
  cursor: pointer;
  transition: background 0.15s;
 }

 .day-number {
  position: relative;
  z-index: 2;
  font-size: 11px;
  color: var(--text-muted);
  line-height: 1.2;
  margin-bottom: 2px;
 }

 .cal-day.today .day-number { color: #fff; }
```

Keep all existing today/selected/has-booking classes exactly as they are.

Remove or empty out .booking-markers and .booking-marker rules since stripes will no longer be rendered inside the cells. Keep .booking-marker-more as-is.

Add a new rule for the continuous bar elements:

```
 .cal-booking-bar {
  position: absolute;
  height: 5px;
  border-radius: 0;
  pointer-events: auto;
  cursor: pointer;
  transition: opacity 0.15s ease;
  z-index: 1;
 }

 .cal-booking-bar:hover { opacity: 0.8; }

 .cal-booking-bar.bar-start { border-radius: 3px 0 0 3px; }
 .cal-booking-bar.bar-end   { border-radius: 0 3px 3px 0; }
 .cal-booking-bar.bar-solo  { border-radius: 3px; }

 @media (max-width: 400px) {
  .cal-booking-bar { height: 4px; }
  .cal-day { min-height: 44px; }
 }

 @media (min-width: 1025px) {
  .cal-booking-bar { height: 6px; }
  .cal-day { min-height: 58px; }
 }
```

STEP 3 — In admin.js, replace the booking marker rendering inside renderCalendar() with a new approach.

Find the section that builds .booking-markers and .booking-marker elements inside each day cell. Remove that entire block.

Instead, after the day grid is fully built (after the closing loop), add a call to a new function: renderBookingBars().

The full renderBookingBars() function to add:

```
 function renderBookingBars() {
```

```
const wrapper = document.getElementById('cal-grid-wrapper');
if (!wrapper) return;

// Remove any previously rendered bars
wrapper.querySelectorAll('.cal-booking-bar').forEach(el => el.remove());

const grid = document.getElementById('cal-grid');
const cells = Array.from(grid.querySelectorAll('.cal-day'));
if (!cells.length) return;

const year  = calDate.getFullYear();
const month = calDate.getMonth();
const firstDay = new Date(year, month, 1).getDay();
const lastDay  = new Date(year, month + 1, 0).getDate();

// Build a map from dateStr -> cell index (0-based within the grid,
// offset by the number of filler cells before day 1)
function cellIndex(d) {
  return firstDay + d - 1;
}

function dateToIndex(dateStr) {
  const [y, m, d] = dateStr.split('-').map(Number);
  if (y !== year || m - 1 !== month) return null;
  if (d < 1 || d > lastDay) return null;
  return cellIndex(d);
}

function parseDateLocal(str) {
  const [y, m, d] = str.split('-').map(Number);
  return new Date(y, m - 1, d);
}

const monthStart = new Date(year, month, 1);
const monthEnd   = new Date(year, month + 1, 0);

// Track vertical offset per cell so multiple bookings stack
const cellOffsets = {};

// Sort bookings by start date so overlapping bookings stack consistently
const sorted = [...allBookings].sort((a, b) =>
  a.start_date.localeCompare(b.start_date)
);

const BAR_HEIGHT   = 5;   // matches CSS, overridden by media query at runtime
const BAR_GAP      = 2;
const TOP_OFFSET   = 22;  // space reserved for the day number

sorted.forEach(b => {
  const bStart = parseDateLocal(b.start_date);
  const bEnd   = parseDateLocal(b.end_date);

  // Clamp to visible month
  const visStart = new Date(Math.max(bStart.getTime(), monthStart.getTime()));
  const visEnd   = new Date(Math.min(bEnd.getTime(),   monthEnd.getTime()));

  if (visStart > visEnd) return;

  const reg = b.is_rented_vehicle
    ? (b.rented_vehicle_reg || null)
    : b.assigned_vehicle_reg;
```

```
const color = getVehicleColor(reg);

// Break the booking into per-row segments (a new row starts every Sunday)
// so bars wrap naturally across week boundaries
let cursor = new Date(visStart);

while (cursor <= visEnd) {
  // This segment runs from cursor to the end of its week row (Saturday)
  // or visEnd, whichever comes first
  const dayOfWeek = cursor.getDay(); // 0=Sun
  const daysUntilSat = 6 - dayOfWeek;
  const segEnd = new Date(Math.min(
    visEnd.getTime(),
    new Date(cursor.getFullYear(), cursor.getMonth(),
          cursor.getDate() + daysUntilSat).getTime()
  ));

  const segStartStr = `${cursor.getFullYear()}-${String(cursor.getMonth()+1).padStart(2,'0')}-${String(cursor.getDate()).padStart(2,'0')}`;
  const segEndStr   = `${segEnd.getFullYear()}-${String(segEnd.getMonth()+1).padStart(2,'0')}-${String(segEnd.getDate()).padStart(2,'0')}`;

  const startIdx = dateToIndex(segStartStr);
  const endIdx   = dateToIndex(segEndStr);

  if (startIdx !== null && endIdx !== null &&
      startIdx < cells.length && endIdx < cells.length) {

    const startCell = cells[startIdx];
    const endCell   = cells[endIdx];

    if (startCell && endCell) {
      const wrapperRect = wrapper.getBoundingClientRect();
      const startRect   = startCell.getBoundingClientRect();
      const endRect     = endCell.getBoundingClientRect();

      const left  = startRect.left  - wrapperRect.left + 2;
      const width = (endRect.right  - wrapperRect.left - 2) - left;

      // Determine stack slot: use the max offset of all cells in range
      let slot = 0;
      for (let i = startIdx; i <= endIdx; i++) {
        slot = Math.max(slot, cellOffsets[i] || 0);
      }
      for (let i = startIdx; i <= endIdx; i++) {
        cellOffsets[i] = slot + 1;
      }

      const top = TOP_OFFSET + slot * (BAR_HEIGHT + BAR_GAP);

      // Determine rounding class
      const isStart = segStartStr === b.start_date ||
                      parseDateLocal(b.start_date) <= monthStart;
      const isEnd   = segEndStr   === b.end_date   ||
                      parseDateLocal(b.end_date)   >= monthEnd;

      const bar = document.createElement('div');
      bar.className = 'cal-booking-bar' +
        (isStart && isEnd ? ' bar-solo' :
          isStart         ? ' bar-start' :
          isEnd           ? ' bar-end'   : '');
      bar.style.cssText = `
```

```
          left:   ${left}px;
          top:    ${top}px;
          width:  ${width}px;
          background: ${color};
        `;
        bar.title = `${b.client_name} (${reg || 'No vehicle'})`;
        bar.addEventListener('click', () => {
          const dateStr = segStartStr;
          const cell    = startCell;
          showBookingsForDate(dateStr, cell);
        });

        wrapper.appendChild(bar);
      }
    }

    // Advance cursor to next Monday (start of next row)
    cursor = new Date(segEnd);
    cursor.setDate(cursor.getDate() + 1);
  }
 });
}
```

STEP 4 — renderBookingBars() reads getBoundingClientRect() which requires the DOM to be painted. Call it after the grid is built:

Inside renderCalendar(), at the very end (after all .cal-day elements are appended to the grid), add:

```
requestAnimationFrame(() => renderBookingBars());
```

Also call renderBookingBars() again whenever the window resizes, so bar positions stay accurate on orientation change or window resize:

```
window.addEventListener('resize', () => {
  if (document.getElementById('tab-calendar')?.classList.contains('active')) {
    requestAnimationFrame(() => renderBookingBars());
  }
});
```

Add this listener once at the bottom of admin.js (outside any function).

STEP 5 — inside the existing day cell loop in renderCalendar(), remove the entire block that creates .booking-markers and appends .booking-marker children. Also remove the MAX_DOTS cap and the .booking-marker-more logic from inside the cell loop — those are no longer needed since the bars are now drawn outside the cells.

Keep the has-booking class toggle exactly as it is:
```
if (dayBookings.length > 0) { el.classList.add('has-booking'); }
```

Keep the el.addEventListener('click', ...) exactly as it is.

== WHAT MUST NOT CHANGE ==

- getVehicleColor() and vehicleColorMap
- calendarMap building logic (the forEach over allBookings that populates it)
- allBookings array and its Supabase query
- showBookingsForDate() function and the day-bookings panel below the grid
- loadCalendarStats() and the stat cards
- cal-nav, cal-prev, cal-next, cal-month-label

- The today, selected, has-booking, other-month CSS classes and their styling
- All other tabs, modals, forms, and pages in the entire app
- All Supabase queries, auth, service worker, offline sync
- The sidebar, header, toast system
- The .booking-paid and .booking-unpaid classes used in the day panel

== VERIFICATION CHECKLIST ==

After changes, confirm each point:

[ ] A booking spanning Mon-Fri shows one continuous coloured bar across all
5 day cells with no gap between them
[ ] A booking spanning Fri-Tue correctly breaks into two row segments:
one bar Fri-Sat on row 1, another bar Sun-Tue on row 2
[ ] Two bookings on the same day stack as two separate bars vertically,
both visible, neither overlapping
[ ] The day number is visible above all bars on every cell
[ ] Clicking anywhere on a bar fires showBookingsForDate for that booking's
start date (or the visible segment start if it wraps)
[ ] The today cell still shows its highlight border and white day number
[ ] Clicking a day cell (not a bar) still opens the booking panel below
[ ] Calendar re-renders correctly after clicking cal-prev and cal-next
[ ] On a 375px screen bars are 4px tall and cells are at least 44px tall
[ ] On a 1440px screen bars are 6px tall and cells are at least 58px tall
[ ] No other page, tab, modal or component has changed appearance
[ ] Rotating an iPad from portrait to landscape re-positions bars correctly